SRU	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: N.pranitha Reg. No:2503a52461
	Department: Computer Science and AISemester: I
	AI Assisted Coding-Assignment Date: 6-10-2026

1. Question:

   Legacy code:

   a=80

   b=70

   c=90

   x=(a+b+c)/3

   if x>=50:

   print("pass")

   else:

   print("fail")

   Student Requirements

   Ask AI to use meaningful variable names.

   Create a function such as calculate_average().

   Improve formatting and readability.

   Run both versions and verify the same result.

   Explain the improvements.

   Submit: Original Code → Prompt → AI Suggestion → Refactored Code → Test Output → Explanation.

2. Prompt: Refactor this code using meaningful variable names and a function such as calculate_average(). Improve formatting and readability, preserve its behavior,

and explain the changes

3. Code:

```
def run_refactored_version():
    scores = [80, 70, 90]
    average_score = calculate_average(scores)
    if average_score >= 50:
        return "pass"
    else:
        return "fail"

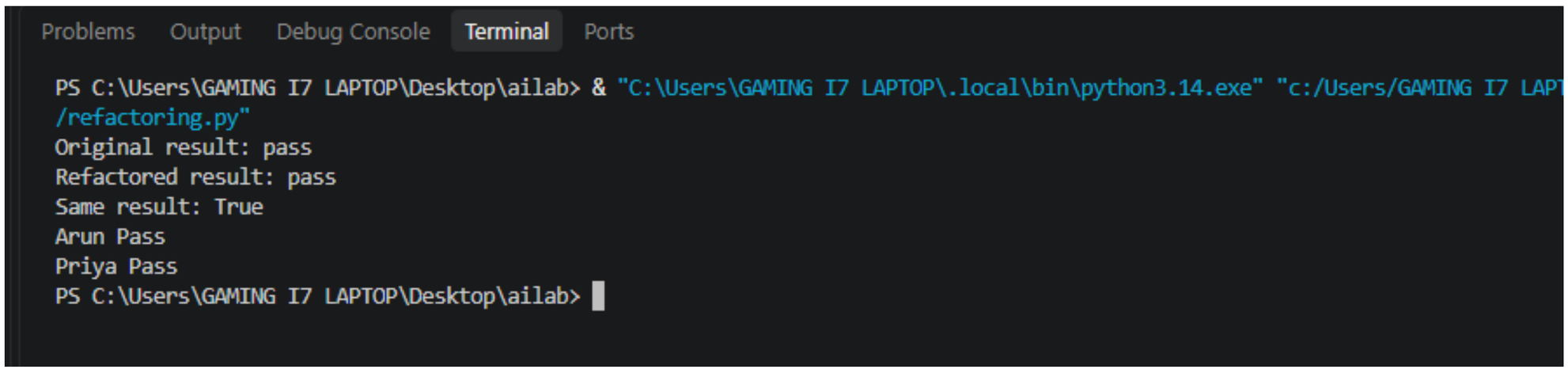

if __name__ == "__main__":
    original_result = run_original_version()
    refactored_result = run_refactored_version()

    print(f"Original result: {original_result}")
    print(f"Refactored result: {refactored_result}")
    print(f"Same result: {original_result == refactored_result}")

    # Task 2
    # Legacy code example converted into a simple reusable function so the
    # same pass/fail logic can be applied to multiple students cleanly.
    students = [
        ("Arun", 85),
        ("Priya", 72),
    ]

    for name, marks in students:
        if marks >= 50:
            print(name, "Pass")
        else:
            print(name, "Fail")
```

4.Output:



5. Observation:

The program was excuted successfully and the output was displayed in the terminal the observed output matches the given progams instructions

2. Question:

Legacy code:

```
name1 ="Arun"
marks1 = 85
if marks1>= 50:
    print(name1, "Pass")
else:
    print(name1, "Fail")

name2 ="Priya"
marks2 = 72
if marks2>= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

Student Requirements

Ask AI to identify duplicated logic.

Refactor the repeated logic into a reusable function.

Test the function with at least four students.

Explain why removing duplication improves maintainability.

Verify that output remains correct.

Submit: Original Code → Prompt → AI Suggestion → Refactored Code → Test Output → Explanation.

Prompt:

Identify the duplicated logic in the given Python code.

Refactor the repeated logic into a reusable function called check_result().

Test the function with at least four students.

Use meaningful variable names and verify that the output remains correct.

Explain how removing duplicate code improves maintainability.

Code:

refactoring.py > ...

```python
def check_result(name, marks):
    if marks >= 50:
        print(name, "Pass")
    else:
        print(name, "Fail")


check_result("Arun", 85)
check_result("Priya", 72)
check_result("Rahul", 40)
check_result("Meena", 55)

```

Output:

```
Refactored result: pass
Same result: True
Arun Pass
Priya Pass
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab> & "C:\Users\GAMING I7 LAPTOP\.local\bin\python3.14.exe" "c:/Use
/refactoring.py"
Arun Pass
Priya Pass
Rahul Fail
Meena Pass
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab>
```

Observation:

The repeated if-else statements were replaced with a reusable function called check_result(). The function was tested with four students. Arun, Priya, and Meena passed, while Rahul failed. The output is correct, and the code is easier to maintain because changes can be made in one function.

3.Question:

Legacy code:

```
age = 68
temperature = 39

if age>= 60 and temperature>= 38:
    print("High Priority")
elif age>= 60 and temperature<38:
    print("Medium Priority")
elif age<60 and temperature>= 38:
    print("Medium Priority")
else:
    print("Normal Priority")
```

Student Requirements

Ask AI to simplify the conditional logic.

Create determine_priority().

Test at least four age/temperature combinations.

Compare original and refactored logic.

Explain whether AI preserved the original behavior.

Submit: Original Code → Prompt → AI Suggestion → Refactored Code → Test Output → Explanation.

Prompt:

Simplify the conditional logic in the given Python code.

Create a function called determine_priority(age, temperature).

Return High Priority when age is at least 60 and temperature is at least 38.

Return Medium Priority when either condition is satisfied.

Otherwise, return Normal Priority.

Test at least four age and temperature combinations.

Compare the original and refactored logic and verify that the original behavior is preserved.

Code:

```python
def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"


print(determine_priority(68, 39))
print(determine_priority(65, 37))
print(determine_priority(45, 39))
print(determine_priority(40, 37))
```

Output:

```
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab> & C:\Users\GAMING I7 LAP
/refactoring.py"
High Priority
Medium Priority
Medium Priority
Normal Priority
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab>
```

Observation:

The long conditional was simplified by combining two conditions using and and or. A reusable function, determine_priority(), was created and tested with four combinations. The output matches the original logic for these cases, making the code easier to understand and maintain.

4.Question:

Legacy code:

patient_ids = [101, 205, 309, 412, 518, 623]

```
target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

Student Requirements

Ask AI to improve readability.

Ask whether a set or dictionary is better for frequent lookups.

Implement the justified suggestion.

Test existing and missing IDs.

Compare typical list and set lookup complexity.

Submit: Original Code → Prompt → AI Suggestion → Refactored Code → Test Output → Explanation.

Prompt:

Improve the readability and efficiency of the given Python code.

Determine whether a set or dictionary is better than a list for frequent membership lookups.

Implement the most suitable solution and test both existing and missing patient IDs.

Compare the typical time complexity of list and set lookups.

Explain the improvements made.

Code:

```
patient_ids = {101, 205, 309, 412, 518, 623}


def find_patient(target_id):
    if target_id in patient_ids:
        print("Patient Found")
    else:
        print("Patient Not Found")


find_patient(412)
find_patient(999)
```

Output:

```
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab> & "C:\Users\GAMING I7 LAP
/refactoring.py"
Patient Found
Patient Not Found
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab>
```

Observation:

The patient IDs were stored in a set instead of a list. A set provides average-case O(1) membership lookup, while a list requires O(n) time in the typical case. The program was tested with an existing ID (412) and a missing ID (999). Both results were correct, and the lookup operation became more efficient for frequent searches.

5.Question:

Legacy code:

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]

total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

Student Requirements

Ask AI to identify at least five improvements.

Refactor using meaningful names and reusable functions.

Use suitable Python features when they improve clarity.

Run both versions and verify equivalent results.

Record AI suggestions as accepted/rejected with reasons.

Identify the most useful refactoring change and justify it.

Submit: Original Code → Prompt → AI Suggestion → Refactored Code → Test Output → Explanation.

Prompt:

Review the given Python code and identify at least five possible improvements.

Refactor the program using meaningful variable names and reusable functions.

Use suitable Python features when they improve readability.

Calculate the average marks and display Pass or Fail for each student.

Run both the original and refactored versions and verify equivalent results.

List the AI suggestions accepted or rejected with reasons.

Identify the most useful refactoring change and justify your choice.

Code:

```
students = [
    ("Arun", 78),
    ("Priya", 45),
    ("Rahul", 88),
    ("Meena", 52)
]


def calculate_average(students):
    total_marks = sum(marks for name, marks in students)
    return total_marks / len(students)


def display_results(students):
    for name, marks in students:
        if marks >= 50:
            print(name, "Pass")
        else:
            print(name, "Fail")


display_results(students)

average_marks = calculate_average(students)
print("Average =", average_marks)
```

Output:

```
/refactoring.py"
Arun Pass
Priya Fail
Rahul Pass
Meena Pass
Average = 65.75
PS C:\Users\GAMING I7 LAPTOP\Desktop\ailab>
```

Observation:

The original program was improved by using meaningful variable names, reusable functions, the built-in sum() function, and clear formatting. The program calculates the average marks as 65.75 and displays the correct result for each student.